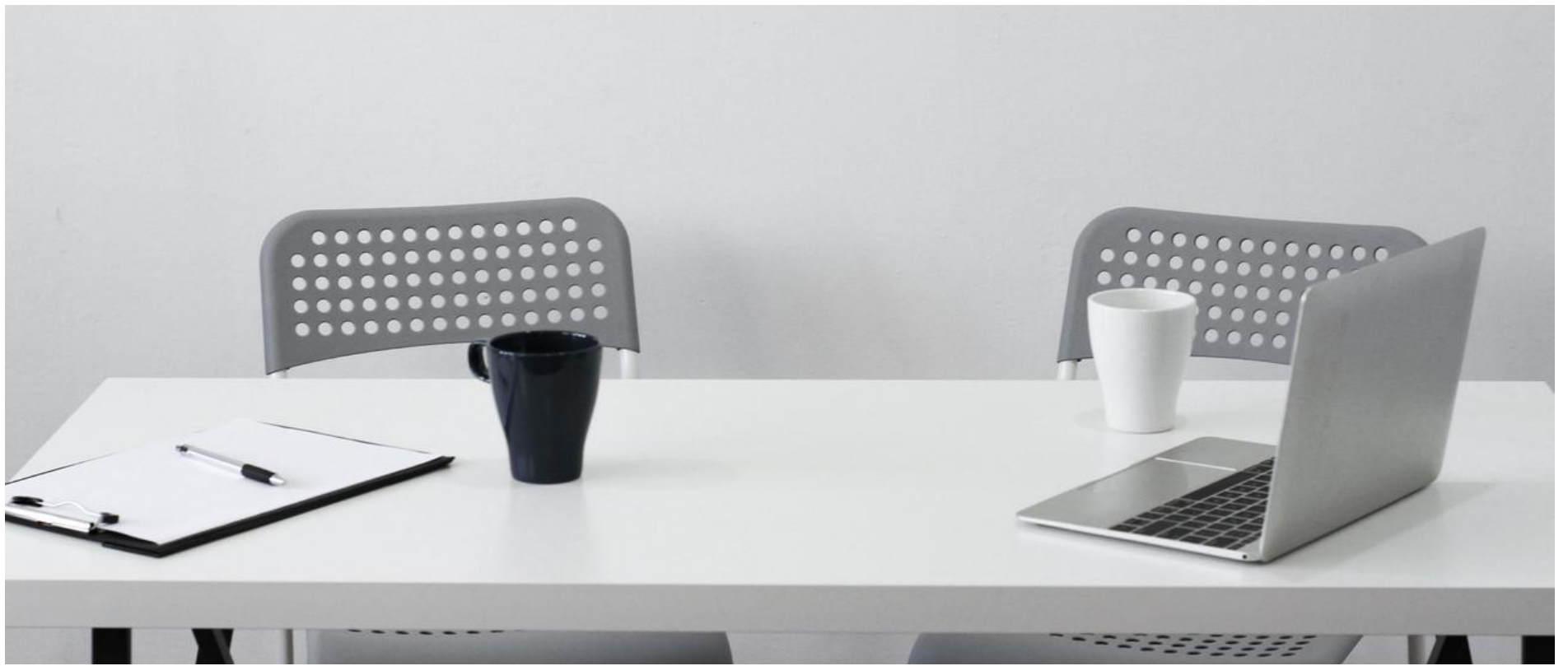




阿里/字节面试高频真题解析与9月考点预测

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 年中复盘会（1/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 年中复盘会（2/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 25年下半年面试真题预测及解析](#)

2. 7~9月面试真题解析

2.1 HTTP/HTTPS

2.1.1 HTTP1.0 1.1 2

HTTP1.0

- 1.0规定浏览器和服务器保持短暂的连接。浏览器的每次请求都需要与服务器建立一个TCP连接，服务器处理完成后立即断开TCP连接（无连接），服务器不跟踪每个客户端也不记录过去的请求（无状态）
 - 无状态，使用cookie
 - 无法复用连接。每次发送请求的时候，都需要进行一次TCP连接，而TCP的连接释放过程又比较费事的。这种无连接的特性会导致网络的利用率非常低。
 - 队头堵塞(head of line blocking)。由于HTTP/1.0规定下一个请求必须在前一个请求响应到达之前才能发送。假设一个请求响应一直不到达，那么下一个请求就不发送，就导致阻塞后面的请求。

2.1.2 HTTPS

对称加密

HTTP1.1

- 持久连接
 - 引入了持久连接，即TCP连接默认不关闭，可以被多个请求复用，不用声明Connection: keep-alive(对于同一个域名，大多数浏览器允许同时建立6个持久连接)
- 管道机制
 - 即在同一个TCP连接里面，客户端可以同时发送多个请求。
- 分块传输编码
 - 即服务端每产生一块数据，就发送一块，采用”流模式”而取代”缓存模式”。
- 新增请求方式
 - PUT:请求服务器存储一个资源;
 - DELETE: 请求服务器删除标识的资源;
 - OPTIONS: 请求查询服务器的性能，或者查询与资源相关的选项和需求;
 - TRACE: 请求服务器回送收到的请求信息，主要用于测试或诊断;
 - CONNECT: 保留将来使用
- 缺点:
 - 虽然允许复用TCP连接，但是同一个TCP连接里面，所有的数据通信是按次序进行的。服务器只有处理完一个请求，才会接着处理下一个请求。如果前面的处理特别慢，后面就会有許多请求排队等着。这将导致“队头堵塞”
 - 避免方式：一是减少请求数，二是同时多开持久连接

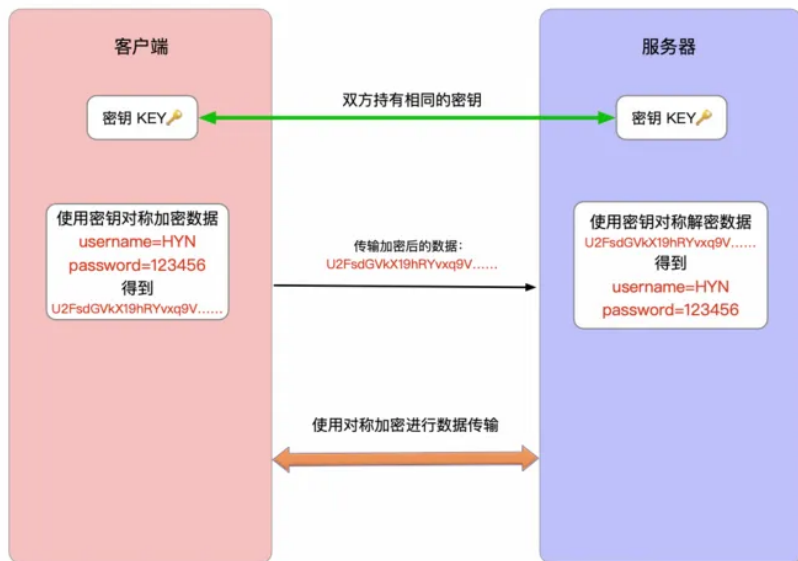
非对称加密

HTTP 2

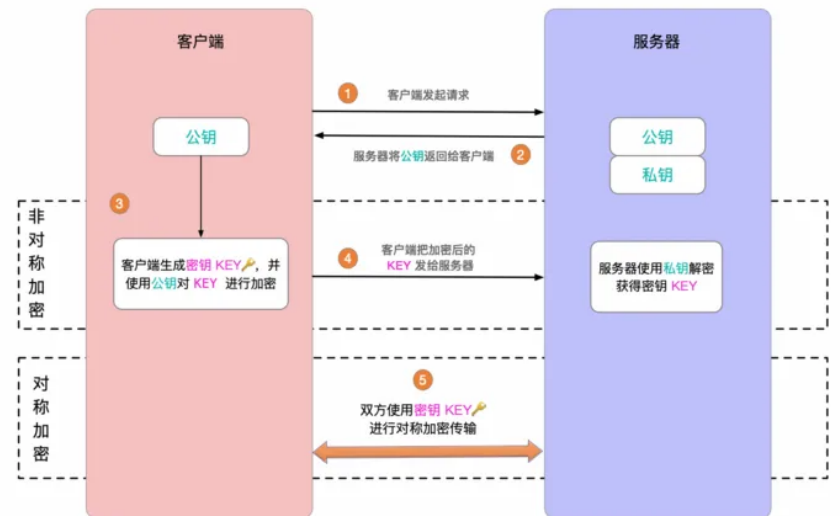
HTTP2

- 二进制协议
 - HTTP/1.1 版的头信息肯定是文本（ASCII编码），数据体可以是文本，也可以是二进制。HTTP/2 则是一个彻底的二进制协议，头信息和数据体都是二进制，并且统称为”帧”：头信息帧和数据帧。
 - 二进制协议解析起来更高效、“线上”更紧凑，更重要的是错误更少。
- 完全多路复用
 - HTTP/2 复用TCP连接，在一个连接里，客户端和浏览器都可以同时发送多个请求或回应，而且不用按照顺序一一对应，这样就避免了”队头堵塞”。
- 报头压缩
 - HTTP 协议是没有状态，导致每次请求都必须附上所有信息。所以，请求的很多头字段都是重复的，比如Cookie，一样的内容每次请求都必须附带，这会浪费很多带宽，也影响速度
 - 对于相同的头部，不必再通过请求发送，只需发送一次;
 - HTTP/2 对这一点做了优化，引入了头信息压缩机制;
 - 一方面，头信息使用gzip或compress压缩后再发送;
 - 另一方面，客户端和服务端同时维护一张头信息表，所有字段都会存入这个表，产生一个索引号，之后就不发送同样字段了，只需发送索引号。
- 服务器推送
 - HTTP/2 允许服务器未经请求，主动向客户端发送资源;
 - 通过推送那些服务器任务客户端将会需要的内容到客户端的缓存中，避免往返的延迟

对称加密数据传输过程



非对称加密发送 KEY 的过程

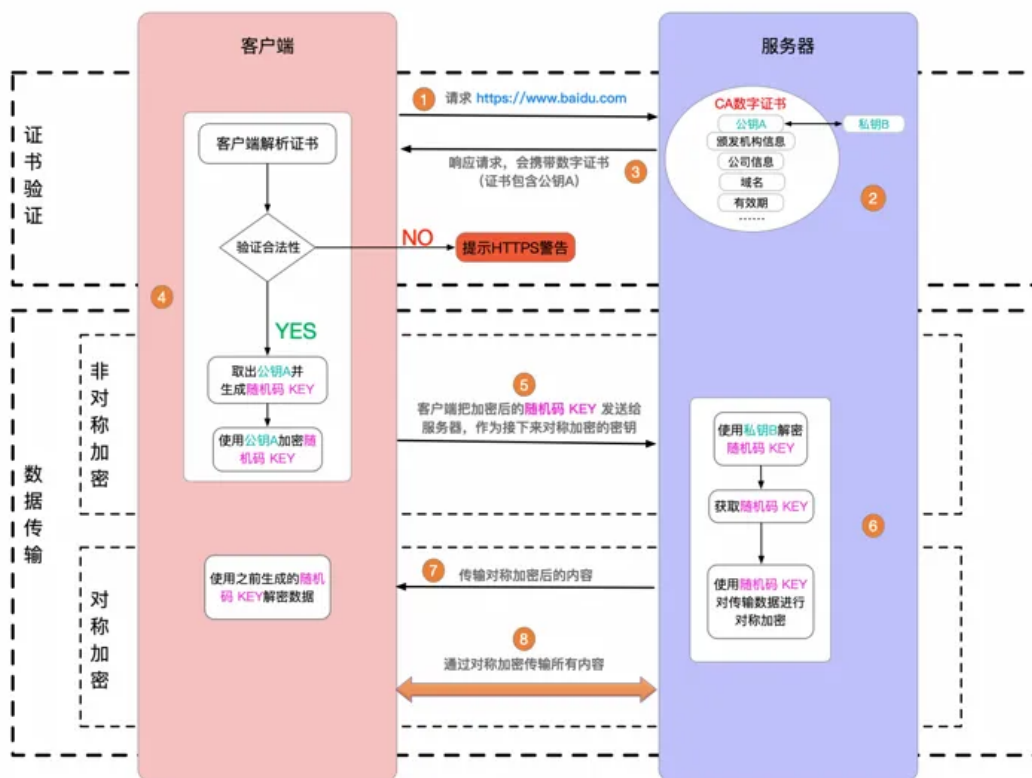


既然双方要使用相同的密钥，那就必然要在传输数据之前先由一方把密钥传给另一方，那么在此过程中密钥就很有可能被截获

HTTPS = HTTP + SSL / TLS。

HTTPS 加密、解密、验证及数据传输过程

by HYN



2.2 跨域

2.2.1 CORS

同源策略是浏览器的一项安全机制，用于限制一个源（origin）的文档或脚本与另一个源的资源进行交互的能力。同源策略的核心规则是：只有当两个URL的协议（protocol）、**域名（host）和端口（port）**都相同时，它们才被认为是同源的。

CORS 的工作原理

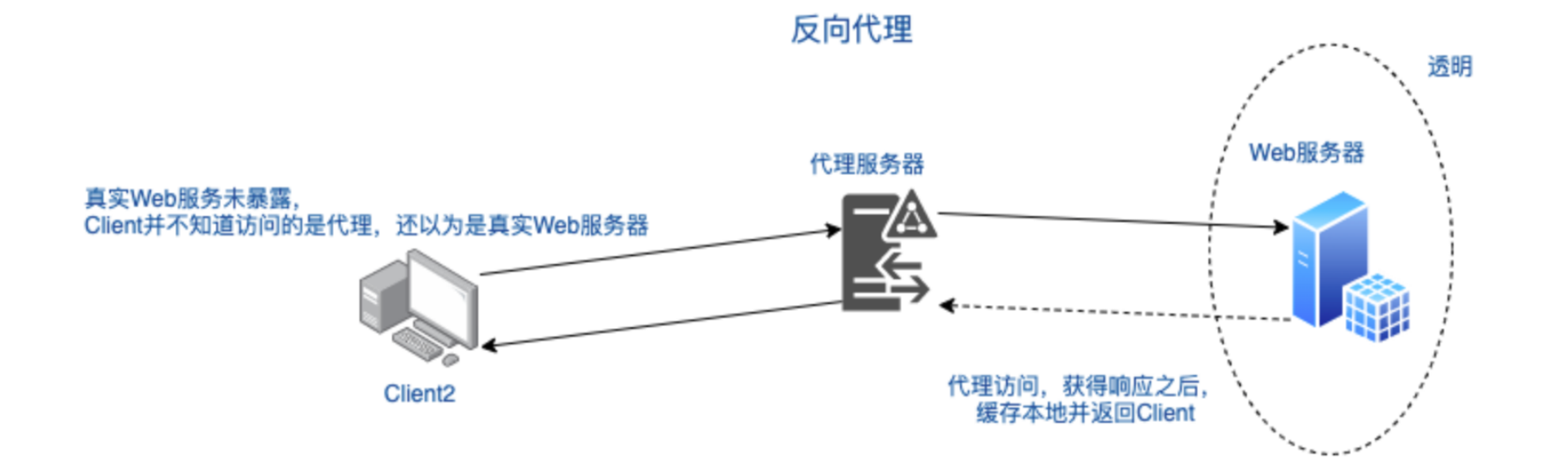
当浏览器发起跨域请求时，它会先发送一个带有 Origin 头的预检请求（preflight request，使用 OPTIONS 方法）。服务器根据请求的 Origin 头决定是否允许该请求，并在响应中返回相应的 CORS 头。

CORS失败原因

a. 预检请求失败：

- 原因：服务器未正确配置 CORS 头。
- 解决方案：确保服务器返回正确的 CORS 头。

b. **CORS 头缺失:**



- a. **负载均衡**：反向代理可以根据后端服务器的负载情况，将请求分发到不同的服务器上，实现负载均衡，提高系统的整体性能。
- b. **安全性增强**：反向代理可以隐藏后端服务器的真实地址和端口，防止直接攻击（如DoS/DDoS）。同时，还可以实现SSL加密、访问控制等安全功能。
- c. **缓存优化**：反向代理可以缓存静态资源，减少后端服务器的负载，提高响应速度。

Code block

```
1  server {
2    listen 80;
3    server_name api.your-domain.com;
4    location / {
5      proxy_pass http://target-server:3000;
6      proxy_set_header Host $host;
7      proxy_set_header X-Real-IP $remote_addr;
8      add_header 'Access-Control-Allow-Origin' 'https://your-domain.com' always;
9    }
10 }
```

2.2.3 本地代理

Code block

```
1  // vite.config.js
2  export default defineConfig({
3    server: {
4      proxy: {
5        '/api': {
6          target: 'http://backend:3000',
7          changeOrigin: true,
8          rewrite: path => path.replace(/^\/api/, '')
9        }
10     }
11   })
12 }
```

2.2.4 JSONP(GET)

Code block

```
1  function jsonp(url, callback) {
2    const script = document.createElement('script');
```



```
3     script.src = `${url}?callback=${callback.name}`;
4     document.body.appendChild(script);
5 }
6
7 function handleResponse(data) {
8     console.log('Received:', data);
9 }
```

2.2.5 PostMessage

Code block

```
1 // 父窗口
2 iframe.contentWindow.postMessage('hello', 'https://child-domain.com');
3
4 // 子窗口监听
5 window.addEventListener('message', event => {
6     if (event.origin !== 'https://parent-domain.com') return;
7     console.log('Received:', event.data);
8 });
```

2.2.6 WebSocket

Code block

```
1 const socket = new WebSocket('wss://your-websocket-server');
2 socket.onopen = () => {
3     socket.send(JSON.stringify({ type: 'ping' }));
4 };
```

2.3 攻击

2.3.1 SQL注入

攻击者通过在输入字段中注入恶意 SQL 代码，从而篡改数据库查询。

防御措施：

- **使用参数化查询：**通过预编译的 SQL 语句和参数化查询，避免直接拼接 SQL。
- **ORM 框架：**使用 ORM 框架（如 Hibernate、Entity Framework）来处理数据库操作。
- **输入验证：**对用户输入进行严格的验证，确保输入符合预期格式。
- **Web 应用防火墙（WAF）：**部署 WAF 来自动检测和过滤恶意 SQL 注入请求。

2.3.2 XSS （Cross-site scripting）

攻击通过在网页中注入恶意脚本，当这些脚本在用户的浏览器中执行时，可能会窃取用户数据或执行恶意操作。

- 存储型 XSS
 - 注入型脚本永久存储在目标服务器上。当浏览器请求数据时，脚本从服务器上传并执行。
- 反射型 XSS
 - 当用户点击一个恶意链接，或者提交一个表单，或者进入一个恶意网站时，注入脚本进入被攻击者的网站。Web 服务器将注入脚本，比如一个错误信息，搜索结果等 返回到用户的浏览器上。由于浏览器认为这个响应来自"可信任"的服务器，所以会执行这段脚本。
- 基于 DOM 的 XSS

- 通过修改原始的客户端代码，受害者浏览器的 DOM 环境改变，导致有效载荷的执行。也就是说，页面本身并没有变化，但由于 DOM 环境被恶意修改，有客户端代码被包含进了页面，并且意外执行

防御措施：

- **输入验证与净化**：对用户输入进行严格的验证和净化，移除潜在的危险字符。
- **输出编码**：在将数据插入 HTML、JavaScript、CSS 或 URL 中时，进行适当的编码。
- **设置 HttpOnly Cookie**：将重要 Cookie 设置为 HttpOnly，防止 JavaScript 访问。

2.3.3 CSRF (Cross-site request forgery) 跨站请求伪造

攻击者诱导受害者进入第三方网站，在第三方网站中，向被攻击网站发送跨站请求。利用受害者在被攻击网站已经获取的注册凭证，绕过后台的用户验证，达到冒充用户对被攻击的网站执行某项操作的目的。

- 一个典型的CSRF攻击有着如下的流程：
 - 受害者登录[a.com](#)，并保留了登录凭证（Cookie）。
 - 攻击者引诱受害者访问了[b.com](#)。
 - [b.com](#) 向 [a.com](#) 发送了一个请求：[a.com/act=xx](#)。浏览器会默认携带[a.com](#)的Cookie。
 - [a.com](#)接收到请求后，对请求进行验证，并确认是受害者的凭证，误以为是受害者自己发送的请求。
 - [a.com](#)以受害者的名义执行了act=xx。
 - 攻击完成，攻击者在受害者不知情的情况下，冒充受害者，让[a.com](#)执行了自己定义的操作。
- 处理：cookie samesite

2.4 Cookie

2.4.1 Secure 和 HttpOnly 属性

Secure 属性和HttpOnly 属性

- Secure：表示只应通过被 HTTPS 协议加密过的请求发送给服务端；
- HttpOnly：JavaScript Document.cookie API 无法访问带有 HttpOnly 属性的 cookie；此类 Cookie 仅作用于服务器。例如，持久化服务器端会话的 Cookie 不需要对 JavaScript 可用，而应具有 HttpOnly 属性。

2.4.2 SameSite

SameSite 可以有下面三种值：

1. None：浏览器会在同站请求、跨站请求下继续发送 cookies，不区分大小写；
2. Strict：浏览器将只在访问相同站点时发送 cookie；
3. Lax：与 Strict 类似，但用户从外部站点导航至 URL 时（例如通过链接）除外。在新版本浏览器中，为默认选项，Same-site cookies 将会为一些跨站子请求保留，如图片加载或者 frames 的调用，但只有当用户从外部站点导航到 URL 时才会发送。如 link 链接；

大多数主流浏览器基本上已经将 SameSite 的默认值迁移至 Lax。如果想要指定 Cookies 在同站、跨站请求都被发送，现在需要明确指定 SameSite 为 None；

2.4.3 安全性

1. 使用 HttpOnly 属性可防止通过 JavaScript 访问 cookie 值；
2. 用于敏感信息（例如指示身份验证）的 Cookie 的生存期应较短，并且 SameSite 属性设置为Strict 或 Lax

2.5 缓存

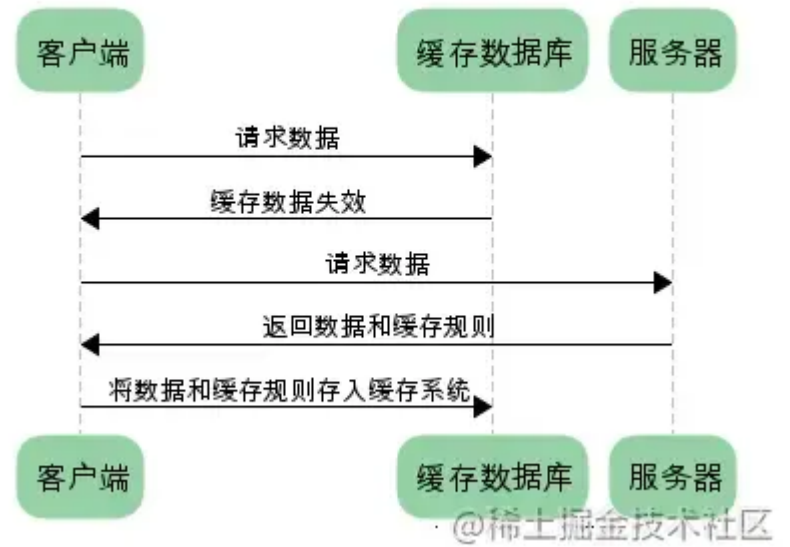
2.5.1 强制缓存

强制缓存不需要与服务器发生交互。

强制缓存规则下，缓存命中



强制缓存规则下，缓存未命中



Expires

HTTP 1.0 的字段，表示缓存到期时间，是一个绝对的时间 (当前时间+缓存时间)。在响应消息头中，设置这个字段之后，就可以告诉浏览器，在未过期之前不需要再次请求。

缺点：若修改电脑的本地时间，会导致浏览器判断缓存失效 这里修重新修改缓存

Cache-control

HTTP/1.1

1. no-cache：告诉浏览器忽略资源的缓存副本，强制每次请求直接发送给服务器，拉取资源，但不是“不缓存”，相当于需要使用协商缓存，禁止使用强制缓存；
2. no-store：强制缓存在任何情况下都不要保留任何副本，相当于不使用强制缓存和协商缓存；
3. public 任何路径的缓存者（本地缓存、代理服务器），可以无条件的缓存改资源，不设置默认为public；
4. private 只针对单个用户或者实体（不同用户、窗口）缓存资源；

2.5.2 对比缓存

last-modified

1. 服务器在响应头中设置last-modified字段返回给客户端，告诉客户端资源最后一次修改的时间；
 - a. `Last-Modified: Sat, 30 Mar 2029 05:46:11 GMT`
2. 浏览器在这个值和内容记录在浏览器的缓存数据库中；
3. 下次请求相同资源，浏览器将在请求头中设置if-modified-since的值（这个值就是第一步响应头中的Last-Modified的值）传给服务器；
4. 服务器收到请求头的if-modified-since的值与last-modified的值比较，如果相等，表示未进行修改，则返回状态码为304；如果不相等，则修改了，返回状态码为200，并返回数据；

Etag

tag是根据文件内容，算出一个唯一的值。服务器存储着文件的Etag 字段。

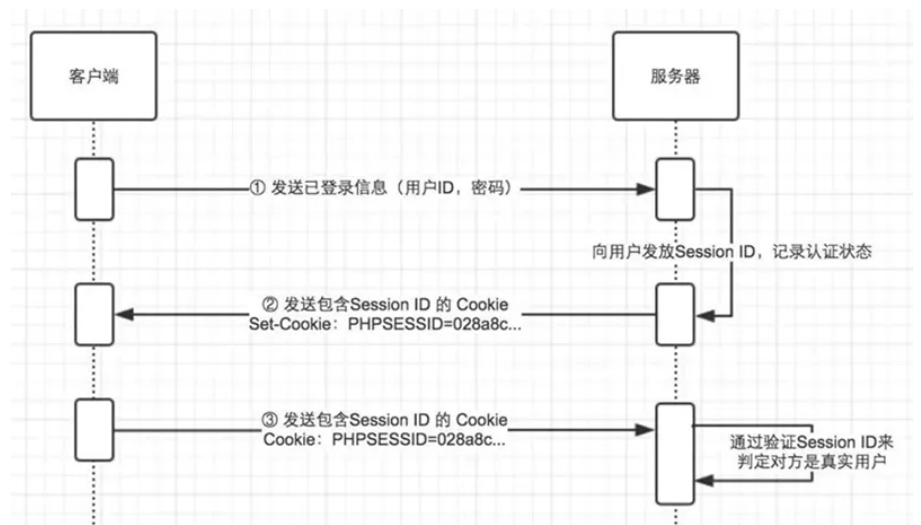
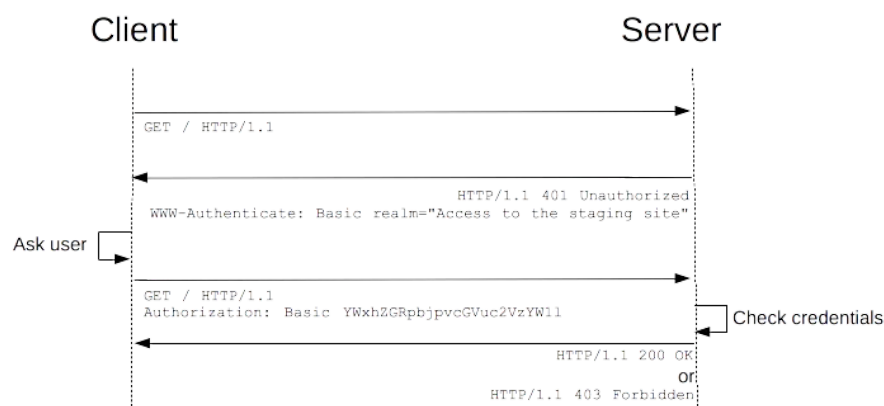
If-None-Match。

服务器同样进行比较，命中返回 304, 不命中返回新资源和 200。 Etag 的优先级高于 Last-Modified

2.6 鉴权

HTTP Basic Authentication

session-cookie



Token JWT (JSON Web Token)

AI智能助手 使用 Authorization

1. 客户端使用用户名跟密码请求登录；
2. 服务端收到请求，去验证用户名与密码；
3. 验证成功后，服务端会签发一个 Token，再把这个 Token 发送给客户端；
4. 客户端收到 Token 以后可以把它存储起来，比如放在 Cookie 里或者 Local Storage 里；
5. 客户端每次向服务端请求资源的时候需要带着服务端签发的 Token；
6. 服务端收到请求，然后去验证客户端请求里面带着的 Token（request头部添加Authorization），如果验证成功，就向客户端返回请求的数据，如果不成功返回401错误码，鉴权失败

跟cookie-session 区别

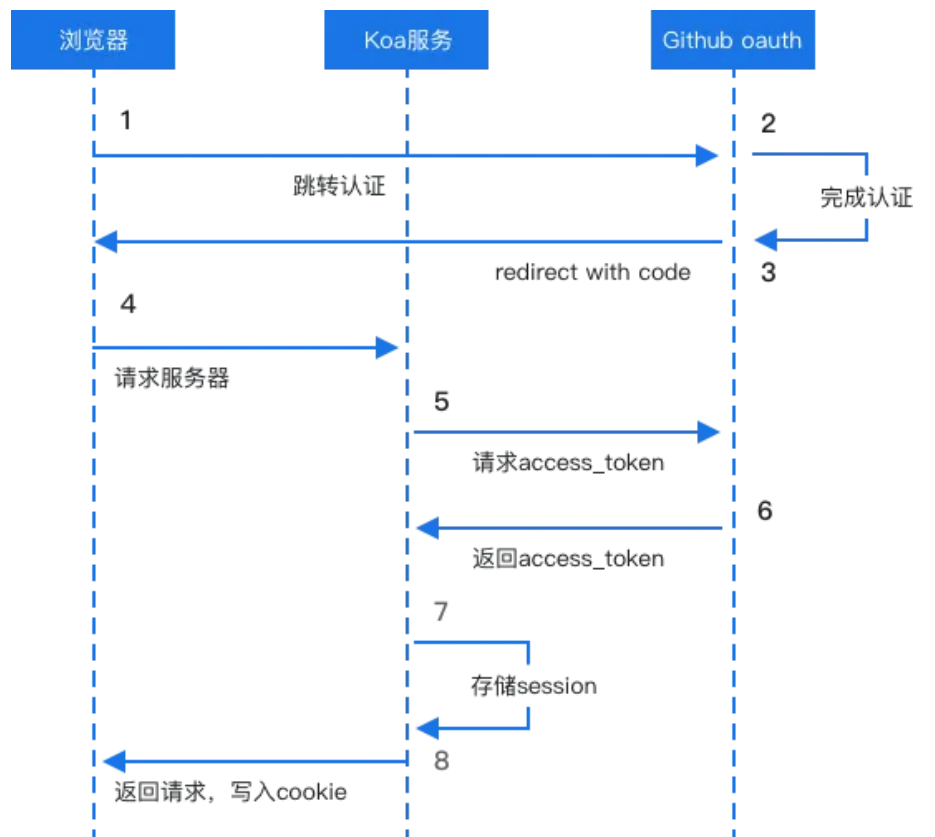
cookie-session

1. 认证方式局限于在浏览器中使用，cookie 是浏览器端的机制，如果在app端就无法使用 cookie；
2. 为了满足全局一致性，我们最好把 session 存储在 redis 中做持久化，而在分布式环境下，我们可能需要在每个服务器上都备份，占用了大量的存储空间；
3. 在不是 Https 协议下使用 cookie，容易受到 CSRF 跨站点请求伪造攻击。

token的缺点：

1. 加密解密消耗使得 token 认证比 session-cookie 更消耗性能；
2. token 比 sid 大，更占带宽；

OAuth



Encoded

PASTE A TOKEN HERE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

{

"alg": "HS256",

"typ": "JWT"

}

PAYLOAD: DATA

{

"sub": "1234567890",

"name": "John Doe",

"iat": 1516239022

}

VERIFY SIGNATURE

HMACSHA256(

base64UrlEncode(header) + "." +

base64UrlEncode(payload),

your-256-bit-secret

)

☐ secret base64 encoded

2.7 水平垂直居中

Code block

1

// 绝对定位 +
translate

2

.parent {

3

position:

relative;

4

}

5

6

.child {

7

position:

absolute;

8

top: 50%;

9

left: 50%;

10

transform:

translate(-50%,

-50%);

11

}

Code block

1

// flex

2

.parent {

3

display: flex;

4

justify-

content:

center; /* 水平居

中 */

5

align-items:

center; /*

垂直居中 */

6

}

Code block

1

// grid

2

.parent {

3

display: grid;

4

place-items:

center; /* 水平垂

直居中 */

5

}

Code block

1

// table cell

2

.parent {

3

display:

table-cell;

4

text-align:

center; /*

水平居中 */

5

vertical-

align: middle;

/* 垂直居中 */

6

}

7

8

.child {

9

display:

inline-block;

10

}

2.8 数组去重

Code block

1

// 1. 使用 Set

2

const uniqueArray = [...new Set(array)];

3

// 2. filter

4

const uniqueArray = array.filter((item, index) => array.indexOf(item) === index);

5

// 3. reduce

6

const uniqueArray = array.reduce((acc, current) => {

7

if (!acc.includes(current)) {

8

acc.push(current);

9

}

10

return acc;

11

}, []);

12

13

// 4. 数组遍历去重

14

const uniqueArray = [];

15

for (let i = 0; i < array.length; i++) {

16

if (uniqueArray.indexOf(array[i]) === -1) {

17

uniqueArray.push(array[i]);

18

}

19

}

2.9 防抖

2.10 节流

在事件被触发n秒后再执行回调，如果在这n秒内又被触发，则重新计时

Code block

```
1 function debounce(func, delay) {
2   let timeoutId;
3   return function(...args) {
4     clearTimeout(timeoutId);
5     timeoutId = setTimeout(() => {
6       func.apply(this, args);
7     }, delay);
8   };
9 }
```

Code block

```
1 import { useCallback, useEffect, useRef } from
  'react';
2
3 function useDebounce(callback, delay) {
4   const timeoutRef = useRef(null);
5
6   const debouncedCallback =
  useCallback((...args) => {
7     if (timeoutRef.current) {
8       clearTimeout(timeoutRef.current);
9     }
10
11     timeoutRef.current = setTimeout(() => {
12       callback(...args);
13     }, delay);
14   }, [callback, delay]);
15
16   useEffect(() => {
17     return () => {
18       if (timeoutRef.current) {
19         clearTimeout(timeoutRef.current);
20       }
21     };
22   }, []);
23
24   return debouncedCallback;
25 }
```

规定在一个单位时间内，只能触发一次函数。如果这个单位时间内触发多次函数，只有一次生效

Code block

```
1 function throttle(func, limit) {
2   let lastFunc;
3   let lastRan;
4   return function(...args) {
5     const context = this;
6     if (!lastRan) {
7       func.apply(context, args);
8       lastRan = Date.now();
9     } else {
10      clearTimeout(lastFunc);
11      lastFunc = setTimeout(() => {
12        if ((Date.now() - lastRan) >= limit) {
13          func.apply(context, args);
14          lastRan = Date.now();
15        }
16      }, limit - (Date.now() - lastRan));
17    }
18  };
19 }
```

Code block

```
1 import { useCallback, useRef } from 'react';
2
3 function useThrottle(callback, delay) {
4   const lastExecuted = useRef(Date.now());
5   const timeoutRef = useRef(null);
6
7   const throttledCallback =
  useCallback((...args) => {
8     const now = Date.now();
9     const timeSinceLastExecution = now -
    lastExecuted.current;
10    const delayRemaining = delay -
    timeSinceLastExecution;
11
12    if (delayRemaining <= 0) {
13      // 足够时间过去了，可以立即执行
14      lastExecuted.current = now;
15      callback(...args);
16    } else if (!timeoutRef.current) {
17      // 设置定时器在剩余时间后执行
18      timeoutRef.current = setTimeout(() => {
19        lastExecuted.current = Date.now();
20        callback(...args);
21        timeoutRef.current = null;
22      }, delayRemaining);
23    }
24  }, [callback, delay]);
25
26  // 清理定时器
27  useEffect(() => {
28    return () => {
29      if (timeoutRef.current) {
30        clearTimeout(timeoutRef.current);
31      }
32    }
33  }, []);
34 }
```

```
32     };
33     }, []);
34
35     return throttledCallback;
36 }
```

3. Vue

3.1 Vue2 Vue3双向绑定 对比

Vue 2 通过 `Object.defineProperty`

- 监听不到对象属性的删除和添加，需要借助 `$set` 和 `$delete` 方法。
- 对于数组的API方法无法监听，vue2中对于数组的方法进行了包裹。
- 数据变化时通知依赖的Watcher 更新视图，需要递归遍历对象来设置getter 和setter，浪费性能。

Vue 3 采用Proxy 监听整个对象，无需递归

3.2 Vue3 比 Vue2 多了什么

- 性能提升:
 - 优化虚拟DOM (Virtual DOM) Vue 3 对虚拟DOM进行了优化，例如静态节点提升、静态props优化等，减少了不必要的DOM操作，提高了渲染性能
 - Vue 3 Proxy -> Vue 2 的Object.defineProperty
 - 碎片(Fragments) Vue 3 支持碎片，允许组件返回多个根节点，也减少了不必要的DOM层级，优化了渲染性能
 - Vue 3 支持tree shaking
 - vue 3 静态标记基础上，支持最长递增子序列
- 响应式系统:
 - Vue 3 Proxy -> Vue 2 的Object.defineProperty
- 代码组织:
 - Vue 3 引入了Composition API，允许开发者更好地组织和复用组件逻辑，使得代码更易于维护和测试。
 - 源码层面上优化 Vue 3 采用模块化设计，将各个组件功能分离，增强了应用程序的灵活性和可扩展性。
- TypeScript 支持:
 - Vue 3 原生支持TypeScript

3.4 Ref reactive 区别

3.3 vue生命周期

1. 创建(Creation):
 - `beforeCreate` : 在实例被创建之初，数据观测(data observer) 和事件(event/watcher) 都尚未初始化。
 - `created` : 实例已经创建完成，数据观测(data observer)、属性和方法的运算，以及watch/event 事件回调都已设置完成，但还没有挂载到DOM 上。
2. 挂载(Mounting):
 - `beforeMount` : 在模板编译/渲染成DOM 之前被调用。
 - `mounted` : 实例已经挂载到DOM 上，此时可以访问真实的DOM 节点。
3. 更新(Updating):
 - `beforeUpdate` : 在数据发生变化，DOM 重新渲染之前被调用。
 - `updated` : 组件DOM 已经更新完成。
4. 销毁(Destruction):
 - `beforeDestroy` : 实例销毁之前调用。在这一步，实例仍然完全可用。
 - `destroyed` : 实例销毁后调用。调用后，Vue 实例的所有指令都解除绑定，所有事件监听器都被移除，所有的子实例也都会被销毁

vue3 新增 `renderTriggered` `renderTracked`

Code block

```
1 // 简化的 reactive 实现
2 function reactive(target) {
3   return new Proxy(target, {
```

特性	ref	reactive
适用数据类型	基本类型和对象	只适用于对象（包括数组）
访问方式	需要通过 <code>.value</code> 访问	直接访问属性
响应式原理	使用对象包装 + getter/setter	使用 Proxy 代理
模板自动解包	自动解包（无需 <code>.value</code> ）	直接使用
重新赋值能力	可以整个替换	不能替换整个对象
TypeScript 支持	更好的类型推断	嵌套属性类型推断较复杂

Code block

```
1 // 简化的 ref 实现
2 function ref(value) {
3   const refObject = {
4     get value() {
5       track(refObject, 'value'); // 依赖收集
6       return value;
7     },
8     set value(newVal) {
9       value = newVal;
10      trigger(refObject, 'value'); // 触发更新
11    }
12  };
13   return refObject;
14 }
```

```
4   get(target, key, receiver) {
5     track(target, key); // 依赖收集
6     return Reflect.get(target, key,
7       receiver);
8   },
9   set(target, key, value, receiver) {
10    Reflect.set(target, key, value,
11      receiver);
12    trigger(target, key); // 触发更新
13    return true;
14  }
```

1. Ref -> reactive: reactive
2. Reactive -> ref: toRefs

使用建议

- 对于简单值，`ref` 比 `reactive` 更轻量
- 对于复杂对象，`reactive` 比深度 `ref` 更高效
- 在模板中使用大量 `ref` 会导致更多 `.value` 访问（虽然模板会自动解包）

3.5 Vue nextTick 实现

在 DOM 更新周期后执行延迟回调，主要基于 JavaScript 的微任务队列机制

Vue 3 使用 Promise 作为 `nextTick` 的实现基础，原因包括1：

1. 微任务优先级高于宏任务（如 `setTimeout`），能更早执行
2. 现代浏览器广泛支持 Promise
3. 可以形成良好的 Promise 链式调用

Code block

```
1 const resolvedPromise = Promise.resolve() as
2   Promise<any>
3 let currentFlushPromise: Promise<void> | null
4   = null
5
6 export function nextTick<T = void>(<
7   this: T,
8   fn?: (this: T) => void
9   ): Promise<void> {
10   const p = currentFlushPromise ||
11     resolvedPromise
12   return fn ? p.then(this ? fn.bind(this) :
13     fn) : p
14 }
```

Vue 2 中 `nextTick` 的实现更加复杂，有一个降级策略：

1. 优先使用 Promise
2. 不支持 Promise 时回退到 MutationObserver
3. 再不支持则使用 setImmediate
4. 最后使用 setTimeout

3.6 Vue 通信

1. 父子组件通信

2. 兄弟组件通信

- **Props**: 父组件通过 `props` 向子组件传递数据。子组件通过 `props` 接收数据。
- `$emit` 和事件 `$on`
- `$parent` 和 `$children`: `$parent` 访问父组件实例, `$children` 访问所有子组件实例。不推荐直接访问组件实例, 因为这会破坏组件的封装性。
- `provide` 和 `inject`
- `$attrs` 和 `$listeners`:

- `eventBus`: 创建一个空的Vue 实例作为事件总线, 通过 `bus.$emit` 触发事件, `bus.$on` 监听事件。这种方式简单, 但适用于小型应用, 对于大型应用, 更推荐使用
- `Vuex`: `Vuex` 是一个专为Vue.js 应用程序开发的状态管理模式。它采用集中式存储管理应用的所有组件的状态, 并以相应的规则保证状态的唯一性。

3. 跨层级组件通信

- `eventBus`: 同上, 适用于小型应用。
- `Vuex`: 同上, 适用于大型应用。
- `provide` 和 `inject`: 同上。

3.7 Vue 状态管理

方案	适用场景	优点	缺点	复杂度	适用规模
组件内状态 (data)	单个组件内部状态	简单直接, 无需额外依赖	难以跨组件共享	低	小型应用
Props/Events	父子组件通信	Vue 原生支持, 简单明了	多层嵌套时繁琐	低	小型应用
Provide/Inject	深层嵌套组件通信	避免 prop 逐层传递	非响应式(默认), 组织性较差	中	中小型应用
Event Bus	任意组件间通信	简单全局通信方案	难以追踪调试, 可能内存泄漏	中	不推荐使用
Vuex	中大型应用集中式状态	集中管理, 调试工具支持, 可预测状态变化	相对繁琐, 学习曲线	高	中大型应用
Pinia	Vue 2/3 应用状态管理	更简单的 API, TypeScript 支持, 模块化	较新, 生态较小	中高	各种规模
Composition API (reactive/ref)	组合式逻辑复用	灵活, 逻辑组合方便	需要良好组织	中	各种规模

4. React

4.1 Hooks 原理

1. **闭包(Closure)**: 使得函数组件在每次渲染之间能够记住和访问之前的状态。
2. **Fiber 架构**: Fiber 节点上会保存一个链表, 用于存储组件中使用的Hooks。这个链表中的每个元素都对应一个Hook, 例如 `useState`, `useEffect` 等。

Fiber

4.2 useState 是同步还是异步

1. `useState` 和 `setState` 看起来是异步的, 但在实际执行上是同步的, 但会进行批处理优化, 以避免频繁的DOM 更新。
2. 在React 合成事件和生命周期钩子函数 (例如 `onClick`、`useEffect`) 中表现为“异步”,

```
code block
1 function FiberNode(
2   tag: WorkTag,
3   pendingProps: mixed,
4   key: null | string,
5   mode: TypeOfMode,
6 ) {
7   // Instance, 静态节点的数据结构属性
8   this.tag = tag;
9   this.key = key;
10  this.elementType = null;
11  this.type = null;
12  this.stateNode = null;
13
14  // Fiber, 用来链接其他fiber节点形成的fiber树
15  this.return = null;
16  this.child = null;
17  this.sibling = null;
18  this.index = 0;
19
20  this.ref = null;
21
22  // 作为动态的工作单元的属性
23  this.pendingProps = pendingProps;
24  this.memoizedProps = null;
25  this.updateQueue = null;
26  this.memoizedState = null;
27  this.dependencies = null;
28 }
```

3. 在原生事件和 `setTimeout`、`Promise` 中的 `then`` 等回调函数中表现为同步

4.3 Vue React diff 复杂度

都是O(n)的时间空间复杂度

4.4 React 状态管理

状态管理库	核心特点	优点	缺点	适用场景
Redux	Flux 架构，单向数据流，不可变状态	- 可预测的状态管理 - 强大的调试工具（Redux DevTools） - 丰富的中间件生态（如 Redux Thunk/Saga）	- 模板代码多，配置繁琐 - 学习成本较高 - 性能优化需手动处理	大型复杂应用，需要严格状态控制的项目
Zustand	轻量级，基于 Hook，类似 Pinia 设计	- 极简 API，无需 Provider - 高性能，按需更新 - 支持中间件（如 Immer）	- DevTools 需额外配置 - 社区生态较新	中小型项目，快速开发，替代 Redux 的场景
MobX	响应式编程，可变状态	- 代码简洁，自动依赖追踪 - 适合 OOP 风格开发	- 调试困难（可变状态） - 异步更新可能导致意外行为	需要灵活状态管理的项目，适合熟悉响应式编程的团队
Context API	React 原生方案，基于 Provider/Consumer	- 无需额外依赖 - 适合简单状态共享	- 性能问题（嵌套 Provider 导致全量渲染） - 不适合复杂状态逻辑	小型应用，解决 Prop Drilling 问题

Jotai	原子化状态管理，类似 Recoil	- 极简 API，按需更新 - 适合细粒度状态控制 - 组合性强	- 依赖 React 上下文，无法在非 React 环境使用	表单、编辑器等需要局部状态管理的场景
Recoil	Facebook 出品，原子化状态	- 精准订阅，避免冗余渲染 - 支持派生状态（Selector）	- 文档较少，社区支持较弱 - 状态依赖链复杂时难维护	实验性项目，需要精细状态控制的场景
Valtio	基于 Proxy 的响应式状态	- 类 Vue 的响应式体验 - 代码量极少	- 可变状态调试困难 - 与 React 设计哲学冲突	快速原型开发，适合熟悉 Vue 的开发者

补充说明：

- 性能对比：Zustand 和 Jotai 在中小型项目中性能表现优异，而 Redux 在大型应用中更稳定。
- 开发体验：Zustand 和 Valtio 的 API 最简洁，适合快速迭代；Redux 适合需要严格架构约束的团队。
- 异步处理：Redux + Redux Thunk/Saga 最成熟，Zustand 和 MobX 也支持但更灵活。

4.5 React 19新特性

新特性

- Actions 用了异步过渡的函数（该函数涉及到异步数据变更）被称作 “Actions”
- useTransition

const [isPending, startTransition] = useTransition()
- useFormStatus

const { pending } = useFormStatus();
- useOptimistic

const [optimisticName, setOptimisticName] = useOptimistic(currentName);
- use
 - 可以异步获取Promise 数据
 - 可以替换 useContext

use 与 Hook 的区别：

- 共同点：use 和 Hook 一样，只能在渲染阶段调用。
 - 区别：use 可以有条件地调用，而 Hook 必须在组件的顶层无条件调用。
- React Server Components（服务器组件）

改进

- 不再需要 forwardRef 就可以直接访问 ref 了
- <Context> 代替 <Context.Provider>
- useDeferredValue支持 initialValue
- 新增对样式表 (stylesheets) 的原生支持，precedence 提前加载
- 错误提示友好
 - onCaughtError：可以用来记录被错误边界捕获的错误，例如发送错误报告到监控系统。
 - onUncaughtError：可以用来处理未被捕获的错误，例如显示一个全局的错误提示，或者执行一些清理操作。
 - onRecoverableError：可以用来记录被自动恢复的错误，例如记录 Hydration 失败的情况

5. 完整的前端技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



6. 9月真题预测 -- webpack 执行流程

1. 初始化阶段 - webpack

- 合并配置项
- 创建 compiler
- 注册插件

2. 编译阶段 - build

- 读取入口文件
- 从入口文件开始进行编译
- 调用 loader 对源代码进行转换
- 借助 babel 解析为 AST 收集依赖模块
- 递归对依赖模块进行编译操作

3. 生成阶段 - seal

- 创建 chunk 对象
- 生成 assets 对象

4. 写入阶段 - emit

6.1 初始化阶段

初始化阶段的逻辑集中在调用 `webpack(config)` 时候，下面我们来看看 `webpack()` 函数体内做了哪些事项。

6.1.1 读取与合并配置信息

通常，在我们的工程的根目录下，会有一个 `webpack.config.js` 作为 `webpack` 的配置来源；

除此之外，还有一种是通过 `webpak bin cli` 命令进行打包时，命令行上携带的参数也会作为 `webpack` 的配置。

在配置文件中包含了我们要让 `webpack` 打包处理的入口模块、输出位置、以及各种 loader、plugin 等；

在命令行上也同样可以指定相关的配置，且 `权重高于` 配置文件。（下面将模拟 `webpack cli` 参数合并处理）

所以，我们在 `webpack` 入口文件这里将先做一件事情：合并配置文件与命令行的配置。

Code block

```
1  // lib/webpack.js
2  function webpack(options) {
3    // 1、合并配置项
4    const mergeOptions = _mergeOptions(options);
5    ...
6  }
7
8  function _mergeOptions(options) {
9    const shellOptions = process.argv.slice(2).reduce((option, argv) => {
10      // argv -> --mode=production
11      const [key, value] = argv.split('=');
12      if (key && value) {
13        const parseKey = key.slice(2);
14        option[parseKey] = value;
15      }
16      return option;
17    }, {});
18    return { ...options, ...shellOptions };
19  }
20
21  module.exports = webpack;
22
```


6.1.2 创建编译器（compiler）对象

好的程序结构离不开一个实例对象，webpack 同样也不甘示弱，其编译运转是由一个叫做 `compiler` 的实例对象来驱动运转。在 `compiler` 实例对象上会记录我们传入的配置参数，以及一些串联插件进行工作的 `hooks` API。同时，还提供了 `run` 方法启动打包构建，`emitAssets` 对打包产物进行输出磁盘写入。这部分内容后面介绍。

Code block

```
1  // lib/webpack.js
2  const Compiler = require('./compiler');
3
4  function webpack(options) {
5    // 1、合并配置项
6    const mergeOptions = _mergeOptions(options);
7    // 2、创建 compiler
8    const compiler = new Compiler(mergeOptions);
9    ...
10   return compiler;
11 }
12
13 module.exports = webpack;
```

`Compiler` 构造函数基础结构如下：

Code block

```
1  // core/compiler.js
2  const fs = require('fs');
3  const path = require('path');
4  const { SyncHook } = require('tapable'); // 串联 compiler 打包流程的订阅与通知钩子
5  const Compilation = require('./compilation'); // 编译构造函数
6
7  class Compiler {
8    constructor(options) {
9      this.options = options;
10     this.context = this.options.context || process.cwd().replace(/\\/g, '/');
11     this.hooks = {
12       // 开始编译时的钩子
13       run: new SyncHook(),
14       // 模块解析完成，在向磁盘写入输出文件时执行
15       emit: new SyncHook(),
16       // 在输出文件写入完成后执行
17       done: new SyncHook(),
18     };
19   }
20
21   run(callback) {
22     ...
23   }
24
25   emitAssets(compilation, callback) {
26     ...
27   }
28 }
29
30 module.exports = Compiler;
```

当需要进行编译时，调用 `compiler.run` 方法即可：

```
Code block
1  compiler.run((err, stats) => { ... });
```

6.1.3 插件注册

有 `compiler` 实例对象后，就可以注册配置文件中的一个插件，在合适的时机来干预打包构建。

插件需要接收 `compiler` 对象作为参数，以此来对打包过程及产物产生 `side effect`。

插件的格式可以是函数或对象，如果为对象，需要自定义提供一个 `apply` 方法。常见的插件结构如下：

```
Code block
1  class WebpackPlugin {
2    apply(compiler) {
3      ...
4    }
5  }
```

注册插件逻辑如下：

```
Code block
1  // lib/webpack.js
2  function webpack(options) {
3    // 1、合并配置项
4    const mergeOptions = _mergeOptions(options);
5    // 2、创建 compiler
6    const compiler = new Compiler(mergeOptions);
7    // 3、注册插件，让插件去影响打包结果
8    if (Array.isArray(options.plugins)) {
9      for (const plugin of options.plugins) {
10        if (typeof plugin === "function") {
11          plugin.call(compiler, compiler); // 当插件为函数时
12        } else {
13          plugin.apply(compiler); // 如果插件是一个对象，需要提供 apply 方法。
14        }
15      }
16    }
17    return compiler;
18  }
```

到这里，webpack 的初始工作已经完成，接下来是调用 `compiler.run()` 进入编译构建阶段。

6.2 编译阶段

编译工作的起点是在 `compiler.run`，它会：

- 1. 发起构建通知，触发 `hooks.run` 通知相关插件；
- 2. 创建 `compilation` 编译对象；
- 3. 读取 entry 入口文件；
- 4. 编译 entry 入口文件；

6.2.1 创建 compilation 编译对象

```

1  // lib/compiler.js
2  class Compiler {
3    ...
4    run(callback) {
5      // 触发 run hook
6      this.hooks.run.call();
7      // 创建 compilation 编译对象
8      const compilation = new Compilation(this);
9      ...
10   }
11 }

```

`compilation` 实例上记录了构建过程中的 `entries`、`module`、`chunks`、`assets` 等编译信息，同时提供 `build` 和 `seal` 方法进行代码构建和代码生成。

Code block

```

1  // lib/compilation.js
2  const fs = require('fs');
3  const path = require('path');
4  const parser = require('@babel/parser');
5  const traverse = require('@babel/traverse').default;
6  const generator = require('@babel/generator').default;
7  const t = require('@babel/types');
8  const { tryExtensions, getSourceCode } = require('./utils');
9
10 class Compilation {
11   constructor(compiler) {
12     this.compiler = compiler;
13     this.context = compiler.context;
14     this.options = compiler.options;
15     // 记录当前 module code
16     this.moduleCode = null;
17     // 保存所有依赖模块对象
18     this.modules = new Set();
19     // 保存所有入口模块对象
20     this.entries = new Map();
21     // 所有的代码块对象
22     this.chunks = new Set();
23     // 存放本次产出的文件对象（与 chunks 一一对应）
24     this.assets = {};
25   }
26   build() {}
27   seal() {}
28 }

```

有了 `compilation` 对象后，通过执行 `compilation.build` 开始模块构建。

Code block

```

1  // lib/compiler.js
2  class Compiler {
3    ...
4    run(callback) {
5      // 触发 run hook
6      this.hooks.run.call();
7      // 创建 compilation 编译对象
8      const compilation = new Compilation(this);
9      // 编译模块
10     compilation.build();

```

```
11    }
12  }
```

6.2.2 读取 entry 入口文件

构建模块首先从 entry 入口模块开始，此时首要工作是根据配置文件拿到入口模块信息。

entry 配置的方式多样化，如：可以不传（有默认值）、可以传入 string，也可以传入对象指定多个入口。

所以读取入口文件需要考虑并兼容这几种灵活配置方式。

Code block

```
1  // lib/compilation.js
2  class Compilation {
3    ...
4    build() {
5      // 1、读取配置入口
6      const entry = this.getEntry();
7      ...
8    }
9
10   getEntry() {
11     let entry = Object.create(null);
12     const { entry: optionsEntry } = this.options;
13     if (!optionsEntry) {
14       entry['main'] = 'src/index.js'; // 默认找寻 src 目录进行打包
15     } else if (typeof optionsEntry === 'string') {
16       entry['main'] = optionsEntry;
17     } else {
18       entry = optionsEntry; // 视为对象，比如多入口配置
19     }
20     // 相对于项目启动根目录计算出相对路径
21     Object.keys(entry).forEach((key) => {
22       entry[key] = './' + path.posix.relative(this.context, entry[key]);
23     });
24     return entry;
25   }
26 }
```

6.2.3 编译 entry 入口文件

拿到入口文件后，依次对每个入口进行构建。

Code block

```
1  // lib/compilation.js
2  class Compilation {
3    ...
4    build() {
5      // 1、读取配置入口
6      const entry = this.getEntry();
7      // 2、构建入口模块
8      Object.keys(entry).forEach((entryName) => {
9        const entryPath = entry[entryName];
10       const entryData = this.buildModule(entryName, entryPath);
11       this.entries.set(entryName, entryData);
12     });
13   }
14 }
15
```

构建阶段执行如下操作：

1. 通过 `fs` 模块读取 entry 入口文件内容；
2. 调用 `loader` 来转换（更改）文件内容；
3. 为模块创建 `module` 对象，通过 AST 解析源代码收集依赖模块，并改写依赖模块的路径；
4. 如果存在依赖模块，递归进行上述三步操作；

读取文件内容：

```
Code block

1  // lib/compilation.js
2  class Compilation {
3    ...
4    buildModule(moduleName, modulePath) {
5      // 1. 读取文件原始代码
6      const originSourceCode = fs.readFileSync(modulePath, 'utf-8');
7      this.moduleCode = originSourceCode;
8      ...
9    }
10 }
```

调用 loader 转换源代码：

```
Code block

1  // lib/compilation.js
2  class Compilation {
3    ...
4    buildModule(moduleName, modulePath) {
5      // 1. 读取文件原始代码
6      const originSourceCode = fs.readFileSync(modulePath, 'utf-8');
7      this.moduleCode = originSourceCode;
8      // 2. 调用 loader 进行处理
9      this.runLoaders(modulePath);
10     ...
11   }
12 }
```

`loader` 本身是一个 JS 函数，接收模块文件的源代码作为参数，经过加工改造后返回新的代码。

```
Code block

1  // lib/compilation.js
2  class Compilation {
3    ...
4    runLoaders(modulePath) {
5      const matchLoaders = [];
6      // 1. 找到与模块相匹配的 loader
7      const rules = this.options.module.rules;
8      rules.forEach((loader) => {
9        const testRule = loader.test;
10       if (testRule.test(modulePath)) {
11         // 如: { test: /\.js$/, use: ['babel-loader'] }, { test: /\.js$/, loader: 'babel-loader' }
12         loader.loader ? matchLoaders.push(loader.loader) : matchLoaders.push(...loader.use);
13       }
14     });
15     // 2. 倒序执行 loader
```



```

16     for (let i = matchLoaders.length - 1; i >= 0; i--) {
17         const loaderFn = require(matchLoaders[i]);
18         // 调用 loader 处理源代码
19         this.moduleCode = loaderFn(this.moduleCode);
20     }
21 }
22 }

```

执行 webpack 模块编译逻辑：

Code block

```

1  // lib/compilation.js
2  class Compilation {
3      ...
4      buildModule(moduleName, modulePath) {
5          // 1. 读取文件原始代码
6          const originSourceCode = fs.readFileSync(modulePath, 'utf-8');
7          this.moduleCode = originSourceCode;
8          // 2. 调用 loader 进行处理
9          this.runLoaders(modulePath);
10         // 3. 调用 webpack 进行模块编译 为模块创建 module 对象
11         const module = this.handleWebpackCompiler(moduleName, modulePath);
12         return module; // 返回模块
13     }
14 }

```

1. 创建 `module` 对象；
2. 对 module code 解析为 `AST` 语法树；
3. 遍历 AST 去识别 `require` 模块语法，将模块收集在 `module.dependencies` 之中，并改写 `require` 语法为 `webpack_require`；
4. 将修改后的 AST 转换为源代码；
5. 若存在依赖模块，深度递归构建依赖模块。

Code block

```

1  // lib/compilation.js
2  class Compilation {
3      ...
4      handleWebpackCompiler(moduleName, modulePath) {
5          // 1、创建 module
6          const moduleId = './' + path.posix.relative(this.context, modulePath);
7          const module = {
8              id: moduleId, // 将当前模块相对于项目启动根目录计算出相对路径 作为模块ID
9              dependencies: new Set(), // 存储该模块所依赖的子模块
10             entryPoint: [moduleName], // 该模块所属的入口文件
11         };
12
13         // 2、对模块内容解析为 AST，收集依赖模块，并改写模块导入语法为 __webpack_require__
14         const ast = parser.parse(this.moduleCode, {
15             sourceType: 'module',
16         });
17
18         // 遍历 ast，识别 require 语法
19         traverse(ast, {
20             CallExpression: (nodePath) => {
21                 const node = nodePath.node;
22                 if (node.callee.name === 'require') {

```

```

23     const requirePath = node.arguments[0].value;
24     // 寻找模块绝对路径
25     const moduleDirName = path.posix.dirname(modulePath);
26     const absolutePath = tryExtensions(
27         path.posix.join(moduleDirName, requirePath),
28         this.options.resolve.extensions,
29         requirePath,
30         moduleDirName
31     );
32     // 创建 moduleId
33     const moduleId = './' + path.posix.relative(this.context, absolutePath);
34     // 将 require 变成 __webpack_require__ 语句
35     node.callee = t.identifier('__webpack_require__');
36     // 修改模块路径 (参考 this.context 的相对路径)
37     node.arguments = [t.stringLiteral(moduleId)];
38
39     if (!Array.from(this.modules).find(module => module.id === moduleId)) {
40         // 在模块的依赖集中记录子依赖
41         module.dependencies.add(moduleId);
42     } else {
43         // 已经存在模块集中。虽然不添加进入模块编译 但是仍要在这个模块上记录被依赖的入口模块
44         this.modules.forEach((module) => {
45             if (module.id === moduleId) {
46                 module.entryPoint.push(moduleName);
47             }
48         });
49     }
50 }
51 },
52 });
53
54 // 3、将 ast 生成新代码
55 const { code } = generator(ast);
56 module._source = code;
57
58 // 4、深度递归构建依赖模块
59 module.dependencies.forEach((dependency) => {
60     const depModule = this.buildModule(moduleName, dependency);
61     // 将编译后的任何依赖模块对象加入到 modules 对象中去
62     this.modules.add(depModule);
63 });
64
65 return module;
66 }
67 }

```

通常我们 require 一个模块文件时习惯不去指定文件后缀，默认会查找 .js 文件。

这跟我们在配置文件中指定的 `resolve.extensions` 配置有关，在 `tryExtensions` 方法中会尝试为每个未填写后缀的 Path 应用 `resolve.extensions`：

Code block

```

1  // lib/utils.js
2  const fs = require('fs');
3
4  function tryExtensions(
5      modulePath, extensions, originModulePath, moduleContext
6  ) {
7      // 优先尝试不需要扩展名选项 (用户如果已经传入了后缀, 那就使用用户填入的, 无需再应用 extensions)
8      extensions.unshift('');

```

```

9   for (let extension of extensions) {
10     if (fs.existsSync(modulePath + extension)) {
11       return modulePath + extension;
12     }
13   }
14   // 未匹配对应文件
15   throw new Error(
16     `No module, Error: Can't resolve ${originModulePath} in ${moduleContext}`
17   );
18 }
19
20 module.exports = {
21   tryExtensions,
22   ...
23 }

```

至此，「编译阶段」到此结束，接下来是「生成阶段」 `seal`。

6.3 生成阶段

在「编译阶段」会将一个个文件构建成 `module` 存储在 `this.modules` 之中。

在「生成阶段」，会根据 `entry` 创建对应 `chunk` 并从 `this.modules` 中查找被 `entry` 所依赖的 `module` 集合。

最后，结合 `runtime` webpack 模块机制运行代码，经过拼接生成最终的 `assets` 产物。

Code block

```

1  // lib/compiler.js
2  class Compiler {
3    ...
4    run(callback) {
5      // 触发 run hook
6      this.hooks.run.call();
7      // 创建 compilation 编译对象
8      const compilation = new Compilation(this);
9      // 编译模块
10     compilation.build();
11     // 生成产物
12     compilation.seal();
13     ...
14   }
15 }

```

`entry + module --> chunk --> assets` 过程如下：

Code block

```

1  // lib/compilation.js
2  class Compilation {
3    ...
4    seal() {
5      // 1、根据 entry 创建 chunk
6      this.entries.forEach((entryData, entryName) => {
7        // 根据当前入口文件和模块的相互依赖关系，组装成为一个个包含当前入口所有依赖模块的 chunk
8        this.createChunk(entryName, entryData);
9      });
10     // 2、根据 chunk 创建 assets
11     this.createAssets();
12   }
13 }

```

```

14 // 根据入口文件和依赖模块组装chunks
15 createChunk(entryName, entryData) {
16   const chunk = {
17     // 每一个入口文件作为一个 chunk
18     name: entryName,
19     // entry build 后的数据信息
20     entryModule: entryData,
21     // entry 的所依赖模块
22     modules: Array.from(this.modules).filter((i) =>
23       i.entryPoint.includes(entryName)
24     ),
25   };
26   // add chunk
27   this.chunks.add(chunk);
28 }
29
30 createAssets() {
31   const output = this.options.output;
32   // 根据 chunks 生成 assets
33   this.chunks.forEach((chunk) => {
34     const parseFileName = output.filename.replace('[name]', chunk.name);
35     // 为每一个 chunk 文件代码拼接 runtime 运行时语法
36     this.assets[parseFileName] = getSourceCode(chunk);
37   });
38 }
39 }

```

`getSourceCode` 是将 `entry` 和 `modules` 组合而成的 `chunk`，接入到 `runtime` 代码模板之中。

Code block

```

1 // lib/utils.js
2 function getSourceCode(chunk) {
3   const { entryModule, modules } = chunk;
4   return `(() => { var __webpack_modules__ = { ${modules.map((module) => {
5     return `    '${module.id}': (module) => {
6       ${module._source}
7     }
8   }
9   }
10  }
11  }
12  }
13  }
14  }
15  }
16  }
17  }
18  }
19  }
20  }
21  }
22  }
23  }
24  }
25  }
26  }
27  }
28  }
29  }
30  }
31  }
32  }
33  }
34  }
35  }
36  }
37  }
38  }
39  }
40  }
41  }
42  }
43  }
44  }
45  }
46  }
47  }
48  }
49  }
50  }
51  }
52  }
53  }
54  }
55  }
56  }
57  }
58  }
59  }
60  }
61  }
62  }
63  }
64  }
65  }
66  }
67  }
68  }
69  }
70  }
71  }
72  }
73  }
74  }
75  }
76  }
77  }
78  }
79  }
80  }
81  }
82  }
83  }
84  }
85  }
86  }
87  }
88  }
89  }
90  }
91  }
92  }
93  }
94  }
95  }
96  }
97  }
98  }
99  }
100 }

```

到这里，「生成阶段」处理完成，这也意味着 `compilation` 编译工作的完成，接下来我们回到 `compiler` 进行最后的「产物输出」。

6.4 写入阶段

「写入阶段」比较容易理解，`assets` 上已经拥有了最终打包后的代码内容，最后要做的就是将代码内容写入到本地磁盘之中。

Code block

```

1 // lib/compiler.js
2 class Compiler {
3   ...
4   run(callback) {

```

```

5    // 触发 run hook
6    this.hooks.run.call();
7    // 创建 compilation 编译对象
8    const compilation = new Compilation(this);
9    // 编译模块
10   compilation.build();
11   // 生成产物
12   compilation.seal();
13   // 输出产物
14   this.emitAssets(compilation, callback);
15 }
16
17 emitAssets(compilation, callback) {
18   const { entries, modules, chunks, assets } = compilation;
19   const output = this.options.output;
20
21   // 调用 Plugin emit 钩子
22   this.hooks.emit.call();
23
24   // 若 output.path 不存在, 进行创建
25   if (!fs.existsSync(output.path)) {
26     fs.mkdirSync(output.path);
27   }
28
29   // 将 assets 中的内容写入文件系统中
30   Object.keys(assets).forEach((fileName) => {
31     const filePath = path.join(output.path, fileName);
32     fs.writeFileSync(filePath, assets[fileName]);
33   });
34
35   // 结束之后触发钩子
36   this.hooks.done.call();
37
38   callback(null, {
39     toJSON: () => {
40       return {
41         entries,
42         modules,
43         chunks,
44         assets,
45       };
46     },
47   });
48 }
49 }

```

至此，webpack 的打包流程就已完成。

6.5 loader

在 `webpack.config.js` 中我们为 `.js` 文件类型配置了一个自定义 loader 来转换文件内容：

Code block

```

1  // webpack.config.js
2  module: {
3    rules: [
4      {
5        test: /\.js/,
6        use: [
7          path.resolve(__dirname, './loaders/transformArrowFnLoader.js'),

```

```
8      ],
9      },
10     ],
11     },
```

loader 本身是一个函数，接收文件模块内容作为参数，经过改造处理返回新的文件内容。

下面我们在 `loaders/transformArrowFnLoader.js` 中，对文件中使用到的箭头函数，转换为普通函数，来理解 `webpack loader` 的作用。

Code block

```
1  // loaders/transformArrowFnLoader.js
2  const parser = require('@babel/parser');
3  const traverse = require('@babel/traverse').default;
4  const generator = require('@babel/generator').default;
5  const t = require('@babel/types');
6
7  function transformArrowLoader(sourceCode) {
8    const ast = parser.parse(sourceCode, {
9      sourceType: 'module'
10    });
11    traverse(ast, {
12      ArrowFunctionExpression(path, state) {
13        const node = path.node;
14        const body = path.get('body');
15        const bodyNode = body.node;
16        if (bodyNode.type !== 'BlockStatement') {
17          const statements = [];
18          statements.push(t.returnStatement(bodyNode));
19          node.body = t.blockStatement(statements);
20        }
21        node.type = "FunctionExpression";
22      }
23    });
24    const { code } = generator(ast);
25
26    return code;
27  }
28
29  module.exports = transformArrowLoader;
30
31
```

6.6 Plugin

从上面介绍我们了解到，每个插件都需要提供一个 `apply` 方法，此方法接收 `compiler` 作为参数。

通过 `compiler` 可以去订阅 `webpack` 工作期间不同阶段的 `hooks`，以此来影响打包结果或者做一些定制操作。

下面我们编写自定义插件，绑定两个不同时刻的 `compiler.hooks` 来扩展 `webpack` 打包功能：

- `hooks.emit.tap` 绑定一个函数，在 `webpack` 编译资源完成，输出写入磁盘前执行（可以做清除 `output.path` 目录操作）；
- `hooks.done.tap` 绑定一个函数，在 `webpack` 写入磁盘完成之后执行（可以做一些静态资源 `copy` 操作）。

Code block

```
1  // plugins/custom-webpack-plugins
2  const fs = require('fs-extra');
3  const path = require('path');
```



```
4
5 class CustomWebpackPlugin {
6   apply(compiler) {
7     const outputPath = compiler.options.output.path;
8     const hooks = compiler.hooks;
9
10    // 清除 build 目录
11    hooks.emit.tap('custom-webpack-plugin', (compilation) => {
12      fs.removeSync(outputPath);
13    });
14
15    // copy 静态资源
16    const otherFilesPath = path.resolve(__dirname, '../src/otherfiles');
17    hooks.done.tap('custom-webpack-plugin', (compilation) => {
18      fs.copySync(otherFilesPath, path.resolve(outputPath, 'otherfiles'));
19    });
20  }
21 }
22
23 module.exports = CustomWebpackPlugin;
```